



NEXBANK ENTERPRISE

High-Security Core Banking Platform

Secure Banking Management System

Comprehensive Technical Specification, System Architecture & End-to-End Implementation Manual

Java 17 LTS

Spring Boot 3.3.4

Spring Security 6

JWT 0.12.6

Hibernate ORM 6.5

MySQL 8.0

React 18

Tailwind CSS 3.4

Vite 8

ACID Transactions

Vaibhav Lakde

Java Full Stack Developer

Pune, Maharashtra, India • vaibhavlakde218@gmail.com • +91 9322963704

GitHub: github.com/vaibhavlakde • LinkedIn: linkedin.com/in/vaibhavlakde-1a4406331

Project Version 1.0.0

Documentation Release: September 2026

1. Executive Summary & Resume Alignment

✓ Verification & Proven Project Alignment

This project strictly fulfills all technical criteria outlined in the **Secure Banking Management System** section of Vaibhav Lakde's professional resume. It demonstrates production mastery across Java 17, Spring Boot, Spring Security 6, Hibernate ORM, MySQL, and modern React frontend design.

The **Secure Banking Management System** (internally designated *NexBank*) is an enterprise-grade full-stack banking application designed with defense-in-depth security, strict ACID financial consistency, and role-based access control. The project showcases end-to-end full-stack software engineering from relational database design and JWT security filters down to interactive component interfaces inspired by modern fintech platforms.

Key Resume Competencies Demonstrated

Resume Specification	Architectural Implementation	Key Technical Artifacts
Login & Role-Based Authentication	Spring Security 6 stateless filter chain, JWT 0.12.6 HMAC-SHA256 signature, BCrypt salting/hashing.	SecurityConfig.java, JwtUtils.java, AuthTokenFilter.java, UserDetailsImpl.java
Account Management REST APIs	Auto-provisioned 10-digit account numbers, IFSC codes (SBMS0001024), real-time beneficiary lookup.	AccountController.java, AccountService.java, AccountRepository.java
Transaction Operations & ACID	Atomic fund transfers with Isolation.REPEATABLE_READ, double-entry ledgering, overdraft protection.	TransactionController.java, TransactionService.java, TransactionRepository.java
Persistent MySQL Management	MySQL 8.0 relational schema mapped via Hibernate 6.5 / Spring Data JPA with foreign key cascade & indexes.	User.java, Account.java, Transaction.java, schema.sql, seed.sql
Responsive React.js Frontend	React 18 Single Page App, Tailwind CSS glassmorphism, virtual debit card, printable receipts, admin portal.	App.jsx, DebitCard.jsx, TransferModal.jsx, AdminPanel.jsx, api.js

2. System Architectural Specification

3. Backend Architecture & Package Structure

The backend is situated inside `/backend` and managed via Apache Maven. It strictly uses Java 17 features including enhanced `switch` expressions, clean records, and modern Spring Boot 3 conventions.

3.1 Core Domain Entities (`com.bank.securebanking.entity`)

- `User.java` : Represents a bank customer or staff member. Maps to the `users` table. Contains `fullName`, unique `email`, BCrypt `password`, `phone`, `role` (`ROLE_CUSTOMER` or `ROLE_ADMIN`), and a one-to-many relationship with `Account`.
- `Account.java` : Represents a financial ledger account. Maps to the `accounts` table. Generates unique 10-digit account numbers (e.g. `1001002026`), holds an IFSC code (`SBMS0001024`), `accountType` (`SAVINGS`, `CURRENT`, `FIXED_DEPOSIT`), current `balance` (`BigDecimal(15,2)`), and `status` (`ACTIVE`, `FROZEN`, `CLOSED`).
- `Transaction.java` : Represents an individual ledger entry. Maps to the `transactions` table. Contains a unique reference UTR number (e.g. `TRF2609030003`), `transactionType` (`DEPOSIT`, `WITHDRAWAL`, `TRANSFER_SEND`, `TRANSFER_RECEIVE`), `amount`, `postBalance` snapshot, and timestamp.

3.2 Security Engine & JWT Infrastructure (`com.bank.securebanking.security`)

The security layer is decoupled and stateless, complying with OWASP financial application guidelines:

JWT Processing Lifecycle

- Authentication:** `AuthController` receives credentials and delegates to `AuthenticationManager` which invokes `UserDetailsServiceImpl`.
- Token Generation:** `JwtUtils.java` encodes subject email, user ID, full name, and expiration time using HMAC-SHA256 with a 256-bit secret key.
- Request Filtering:** For subsequent requests, `AuthTokenFilter.java` extracts `Authorization: Bearer <token>`, checks integrity via `Jwts.parser().verifyWith(key)`, and loads user permissions into the `SecurityContext`.
- Unauthorized Trapping:** If expired or missing, `AuthEntryPointJwt.java` returns a structured JSON 401 error.

```
// Security Configuration Snippet (SecurityConfig.java)
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .cors(cors → cors.configurationSource(corsConfigurationSource()))
        .csrf(csrf → csrf.disable())
        .exceptionHandling(ex → ex.authenticationEntryPoint(unauthorizedHandler))
        .sessionManagement(s → s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth → auth
            .requestMatchers("/api/auth/**", "/api/health", "/h2-console/**").permitAll()
            .requestMatchers("/api/admin/**").hasAnyAuthority("ROLE_ADMIN", "ROLE_BANKER")
            .anyRequest().authenticated()
        );
    http.authenticationProvider(authenticationProvider());
    http.addFilterBefore(authTokenFilter, UsernamePasswordAuthenticationFilter.class);
    return http.build();
}
```

3.3 Transactional Core & Double-Entry Bookkeeping (com.bank.securebanking.service)

All fund movements are handled within `TransactionService.java` under strict ACID guarantees:

```
@Transactional(isolation = Isolation.REPEATABLE_READ)
public TransactionDto transfer(TransferRequest request) {
    // 1. Validation checks: account existence, account equality, active status
    // 2. Overdraft validation: fromAccount.getBalance().compareTo(request.getAmount()) ≥ 0
    // 3. Sender Debit: newBalance = balance - amount
    // 4. Recipient Credit: newBalance = balance + amount
    // 5. Audit Logging: Insert paired double-entry ledger rows (TRANSFER_SEND & TRANSFER_RECEIVE)
    // 6. Return verified TransactionDto with reference UTR
}
```

4. REST API Contract & Endpoints

All REST APIs adhere to uniform request and response contracts wrapped inside the generic `ApiResponse<T>` envelope.

4.1 Authentication Endpoints (/api/auth)

Method & Endpoint	Payload Parameters	Response Data	Description
POST /api/auth/register	fullName, email, password, phone, accountType, initialDeposit	ApiResponse (JWT Token, User ID, Account Number)	Onboards new customer & automatically provisions an active bank account.
POST /api/auth/login	email, password	ApiResponse (JWT Token, User ID, Role)	Validates credentials against BCrypt hash; issues 24-hour Bearer token.
GET /api/auth/me	None (Requires Bearer <token>)	User profile JSON (ID, Full Name, Email, Phone, Role)	Retrieves the current authenticated profile.

4.2 Account Management Endpoints (/api/accounts)

Method & Endpoint	Access Level	Response Data	Description
GET /api/accounts/my-accounts	Customer / Admin	List<AccountDto>	Retrieves all accounts belonging to the authenticated user.
GET /api/accounts/{accountNumber}	Owner / Admin	AccountDto	Retrieves specific balance, IFSC code, and account status.
GET /api/accounts/verify/{accountNumber}	Authenticated	AccountDto (Safe)	Validates recipient account; returns recipient customer name for confirmation.
POST /api/accounts/new	Authenticated	AccountDto	Generates additional Savings, Current, or Fixed Deposit accounts.
PATCH /api/accounts/{accountNumber}/status	ROLE_ADMIN	AccountDto	Updates account lifecycle state (ACTIVE , FROZEN , CLOSED).

4.3 Transaction Operations Endpoints (/api/transactions)

Method & Endpoint	Payload Parameters	Response Data	Description
POST /api/transactions/deposit	accountNumber, amount, description	TransactionDto	Credits funds to active account; records ledger credit entry.
POST /api/transactions/withdraw	accountNumber, amount, description	TransactionDto	Debits funds after balance verification; simulates ATM cash payout.
POST /api/transactions/transfer	fromAccountNumber, toAccountNumber, amount, description	TransactionDto	Atomic P2P transfer; debits sender, credits receiver, logs dual UTRs.
GET /api/transactions/history/{acc}	Account Owner / Admin	List<TransactionDto>	Returns chronological ledger history sorted newest first.
GET /api/transactions/receipt/{ref}	Authenticated	TransactionDto	Retrieves individual transaction receipt by UTR reference code.

4.4 Bank Administration Endpoints (/api/admin)

Method & Endpoint	Required Role	Description
GET /api/admin/dashboard	ROLE_ADMIN	Returns aggregate bank reserves, total customers, accounts, credit/debit volume.
GET /api/admin/accounts	ROLE_ADMIN	Returns complete registry of all customer accounts in the institution.
GET /api/admin/transactions	ROLE_ADMIN	Returns immutable global audit log of all transactions system-wide.

5. Frontend Architecture & Component Hierarchy

The frontend resides in `/frontend`, built on **React 18** and bundled with **Vite 8**. It features responsive utility-first styling with **Tailwind CSS 3.4**, Lucide React icons, and a custom fintech color palette.

5.1 Component Tree & Responsibilities

```
App.jsx (Global State Controller, Modals, Toasts)
├── Navbar.jsx (Branding, Account Balance Dropdown, User Role Tag, Logout)
├── Sidebar.jsx (Navigation: Dashboard, Transfer, Deposit, Withdraw, Admin)
├── DebitCard.jsx (Virtual Platinum Card: EMV Chip, Card Masking, IFSC, Copy)
├── TransferModal.jsx (2-Step Inter-Account Transfer with Real-Time Beneficiary Lookup)
├── DepositWithdrawModal.jsx (Quick-Amount Deposit & ATM Cash Out Operations)
├── TransactionReceiptModal.jsx (Printable / PDF Cryptographic Receipt)
├── AdminPanel.jsx (Reserve Analytics, Account Freeze/Unfreeze Controls, Audit Trail)
└── LoginModal.jsx (1-Click Demo Login Switcher & New Customer Onboarding)
```

5.2 Key Frontend UI Highlights

- **Virtual Platinum Debit Card** (`DebitCard.jsx`): Features metallic radial gradient styling, gold EMV contact chip, contactless RFID icon, toggleable card number masking (`.... 2026`), 1-click clipboard copying, and live account status tag.
- **Two-Step Fund Transfer Flow** (`TransferModal.jsx`):
 1. *Step 1*: User enters recipient account. The **Verify Name** feature queries `/api/accounts/verify/{acc}` to verify the legal account holder (e.g. *Priya Sharma*) before any money is transferred.
 2. *Step 2*: Displays summary review and asks for a 4-digit Security PIN (`1234`) to authorize the transaction.
- **Printable Digital Receipt** (`TransactionReceiptModal.jsx`): Styled with custom `@media print` rules. Users can click **"Print Receipt"** to trigger native browser printing or save directly to PDF.
- **Admin Portal** (`AdminPanel.jsx`): Gives banking supervisors live visibility into total bank reserves, customer distribution, and provides immediate 1-click **Freeze / Unfreeze** buttons for any account.

6. Database Schema & Data Dictionary

The relational database is documented in `/database/schema.sql` and automatically provisioned by Hibernate on startup.

Table Name	Column Name	Data Type	Constraints	Description
users	id	BIGINT	PK, Auto Increment	Primary user surrogate identifier
	full_name	VARCHAR(100)	NOT NULL	Customer legal name
	email	VARCHAR(120)	NOT NULL, UNIQUE	Authentication username
	password	VARCHAR(255)	NOT NULL	BCrypt salted password hash
	role	VARCHAR(30)	NOT NULL	ROLE_CUSTOMER or ROLE_ADMIN
accounts	id	BIGINT	PK, Auto Increment	Account surrogate identifier
	account_number	VARCHAR(20)	NOT NULL, UNIQUE	10-digit public account number
	ifsc_code	VARCHAR(15)	NOT NULL	Branch IFSC (SBMS0001024)
	balance	DECIMAL(15,2)	NOT NULL	Current ledger balance
	status	VARCHAR(20)	NOT NULL	ACTIVE , FROZEN , CLOSED
	user_id	BIGINT	FK → users.id	Cascade delete ownership link
transactions	id	BIGINT	PK, Auto Increment	Transaction surrogate identifier
	reference_number	VARCHAR(36)	NOT NULL, UNIQUE	Bank UTR reference code
	account_id	BIGINT	FK → accounts.id	Associated account ledger
	transaction_type	VARCHAR(25)	NOT NULL	DEPOSIT , WITHDRAWAL , TRANSFER_*
	amount	DECIMAL(15,2)	NOT NULL	Debit or credit transaction amount
	post_balance	DECIMAL(15,2)	NOT NULL	Balance after transaction execution

7. Operational Guide & Pre-Seeded Demo Data

7.1 Pre-Configured Test Accounts

The application automatically seeds realistic data via `DataInitializer.java` on initial boot:

Role / Persona	Login Email	Password	Account Number	Initial Balance	Capabilities
Primary Customer (Vaibhav Lakde)	vaibhav@gmail.com	Vaibhav@123	1001002026	₹75,450.00	Transfers, cash deposits, ATM cash withdrawals, printable receipts, account creation.
Beneficiary Customer (Priya Sharma)	priya@gmail.com	Priya@123	1001002027	₹42,800.00	Receives peer-to-peer transfers, verifies name lookup.
Bank Administrator (System Admin)	admin@bank.com	Admin@123	Central Reserve	Total Reserves	Full bank oversight, customer account freeze/unfreeze toggles, global audit log.

7.2 Running the Application Locally

Step 1: Start Backend (Port 8080)

```
cd backend
mvn clean spring-boot:run
```

Verify health: `http://localhost:8080/api/health`

Step 2: Start Frontend (Port 5173)

```
cd frontend
npm install
npm run dev
```

Open browser: `http://localhost:5173`

8. Technical Interview Preparation & Talking Points

💡 How to present this project during a technical interview

Highlight the architectural decisions, security considerations, and edge cases you tackled rather than just listing features.

Q1: "How did you ensure fund transfers don't result in dirty reads or double-spending?"

Answer: "In `TransactionService`, I annotated the transfer method with `@Transactional(isolation = Isolation.REPEATABLE_READ)`. This ensures that when querying the sender's balance, no concurrent transaction can modify the record during execution. The debit and credit legs occur within the exact same database transaction boundary. If an exception occurs—such as a network timeout or insufficient funds—the entire transaction rolls back cleanly, maintaining ACID compliance."

Q2: "How is security implemented without session state?"

Answer: "I implemented a stateless JWT authentication model using Spring Security 6 and JJWT 0.12.6. When a user logs in, their password is validated using BCrypt. Upon success, a signed HMAC-SHA256 JWT is returned containing their identity and role authorities. An `AuthTokenFilter` subclasses `OncePerRequestFilter` to inspect the Bearer token on every request, validate its cryptographic signature and expiration, and establish the user's `Authentication` in the `SecurityContextHolder`."

Q3: "How does the frontend handle token expiration and authorization errors?"

Answer: "The React frontend uses a centralized API client in `api.js`. If a response returns HTTP 401 Unauthorized or 403 Forbidden, it automatically triggers a logout callback that purges local storage tokens and pops up the `LoginModal`, notifying the user gracefully with a toast notification rather than breaking the UI state."